

백엔드 판단을 AI Native 개발 자산으로

Java·Kotlin 기반 이벤트·배치·검색·캐시 설계·운영. 판단 기준은 에이전트·스킬·평가에 담아 개발 흐름에서 재사용.

서민재 · Rex Seo

AI-Native Backend Engineer

ckdekrn88@gmail.com

GitHub

Velog

LinkedIn

이력서

경력기술서

01

POINT PLATFORM · EVENT RELIABILITY

발행 실패와 처리 실패, 복구 경로 분리

이벤트 레코드에 실패 지점과 복구 대상을 기록. 로그 재추적 없이 포인트 차감·회수 업무만 재실행.

담당 이벤트 상태·수동 복구·정체 알림 설계 및 구현	구조 내부 이벤트 9종, 상태 6개, 타입별 복구 전략 8개	저장 PostgreSQL internal_event_record · JSONB payload	관측 IN_PROGRESS 15분 초과 건을 타입별로 집계해 Slack 전송
상황 단일 ‘실패’ 상태: 원인 탐색부터 재시작 - 발행 실패·소비자 처리 중단: 확인 데이터·재시도 위치가 다름 - 단일 상태: 운영자가 로그·원장을 처음부터 수동 대조	판단 실패 지점 기록과 업무별 복구 로직 분리 - 상태 6개: 생성부터 복구 불가 실패까지 구분 - 포인트 차감·회수·만료 알림 등 작업 8개: 저장된 payload로 각 업무 서비스 재호출		

멈춘 위치에서 다시 시작하는 복구 흐름

생성부터 처리, 수동 복구와 정체 감지까지

호출 비동기 복구

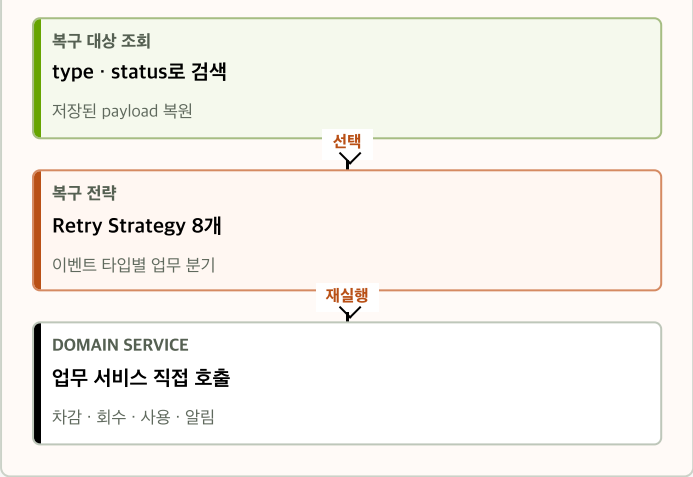
01 정상 처리

업무 요청과 처리 상태를 기록한 뒤 Kafka로 전달합니다.



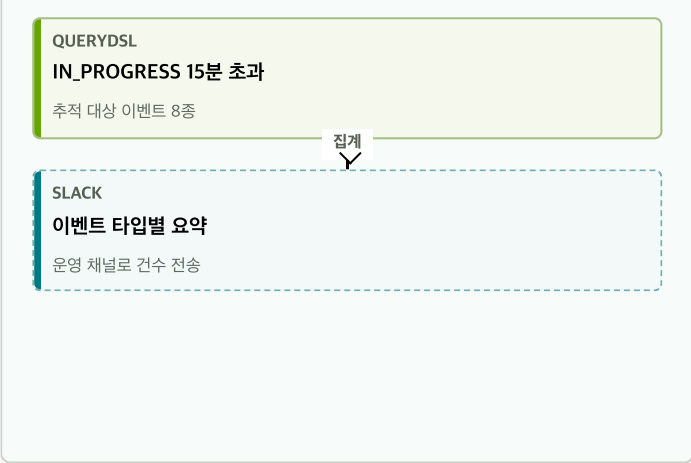
02 수동 복구

Kafka에 다시 넣지 않고 실패한 업무만 실행합니다.



03 정체 감지

15분 넘게 처리 중인 이벤트를 Slack으로 알립니다.



달라진 운영

전송·처리 실패 상태 분리 · IN_PROGRESS 15분 초과 Slack 알림 · 저장 payload 기반
업무 단위 재실행

배치는 재시작 지점, 동시 요청은 결과 불변식

원장 소멸·만료 알림·아카이브별 재시작 기준. 동시 포인트 거래·예산 차감 후 잔액·원장 정합성 검증.

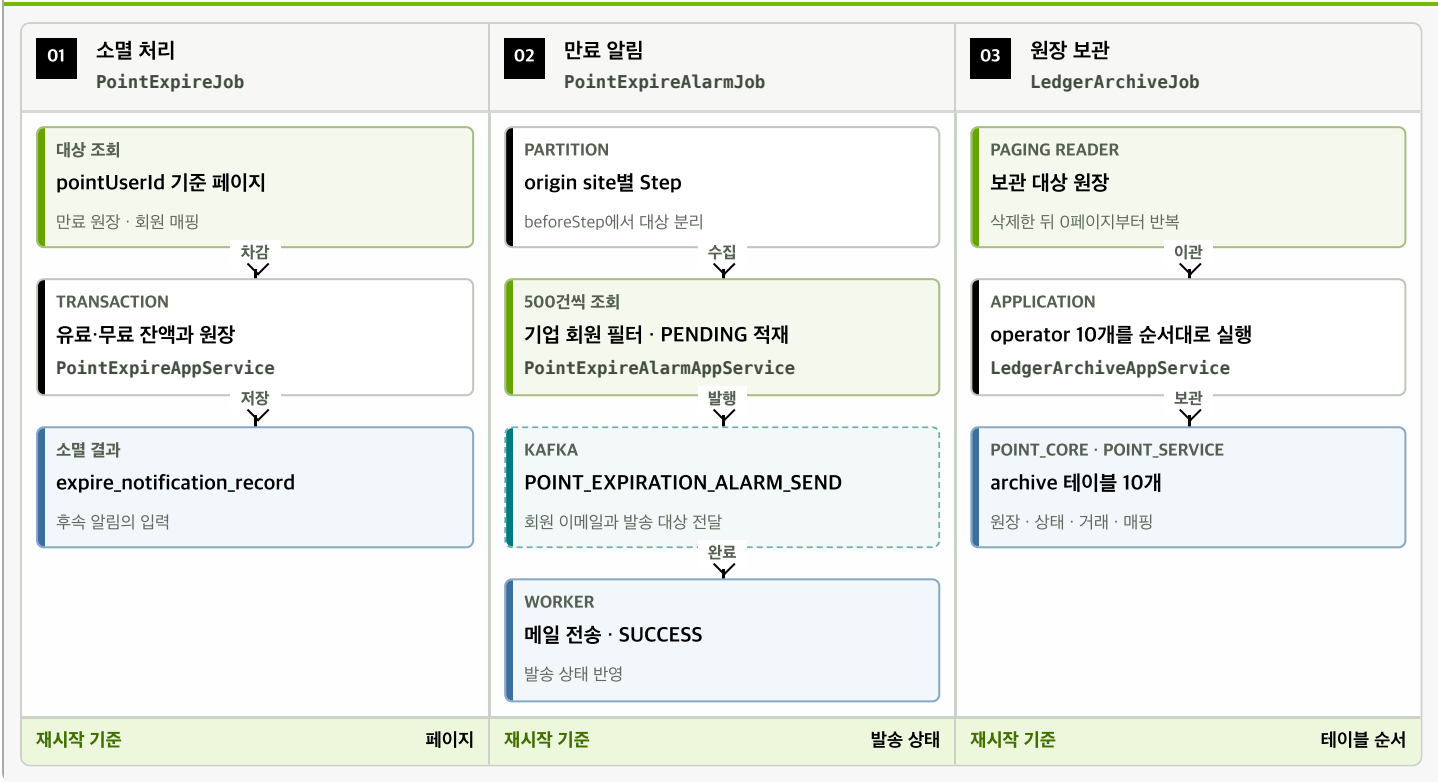
담당 원장 소멸 페이지 전환· 아카이브 배치 구현·만료 알림 이벤트 개선	처리 단위 전체 적재 대신 페이지 단위 반복 처리	보관 대상 point_core 6개·point_service 4개 archive 테이블	동시성 9개 API · 13개 시나리오 · 3·5·10개 동시 요청
--	--	---	--

상황 같은 데이터, 서로 다른 세 Job의 처리 기준 - 소멸: 잔액·원장 변경 - 만료 안내: 알림 적재·발송 - 아카이브: 참조 순서대로 테이블 이관 · 데이터 증가 시 전체 적재 실패 범위 확대	판단 배치는 재시작 위치, 동시 요청은 결과 불변식 - 소멸·알림·아카이브: 진행 위치 기록 - 포인트 거래·예산 차감: 락 획득 여부와 경쟁 종료 후 잔액·원장 상태·이벤트 수 모두 검증
--	--

세 배치, 세 가지 재시작 기준

같은 원장을 다뤄도 다시 시작할 기준은 서로 다릅니다.

— 순차 처리 - - - 후속 이벤트



03

FEED & SEARCH · SERVING / RETRIEVAL

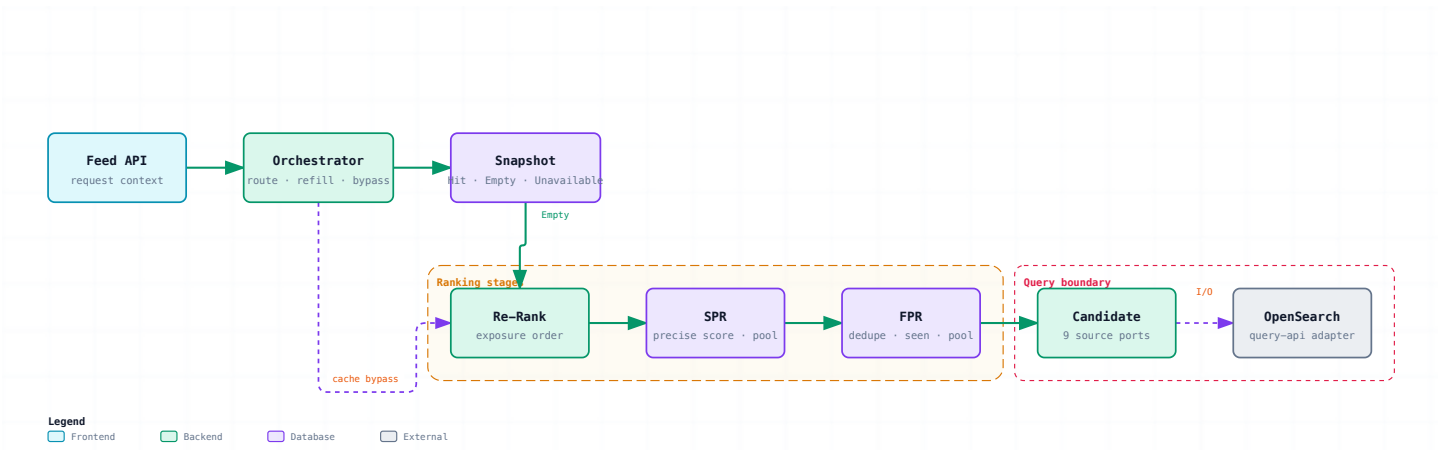
검색·랭킹·서빙 책임 경계

Candidate Retrieval → FPR → SPR → Re-Rank → Snapshot. 랭킹은 pull 계약, OpenSearch·Redis는 포트로 격리.

전체 흐름 Candidate Retrieval → FPR → SPR → Re-Rank → Snapshot	공통 계약 후보 수집 포트 · 역방향 pull 계약 · 캐시 포트 3개	검색 경계 검색 소스 9개 · 소스별 정책 · 타임아웃·실패 격리	모듈 경계 query-api는 OpenSearch 연동 전용 · 순환 의존 금지
---	--	---	---

피드 서빙 책임 경계

Snapshot 상태가 경로를 정하고, 랭킹·검색은 pull 계약 뒤로 분리

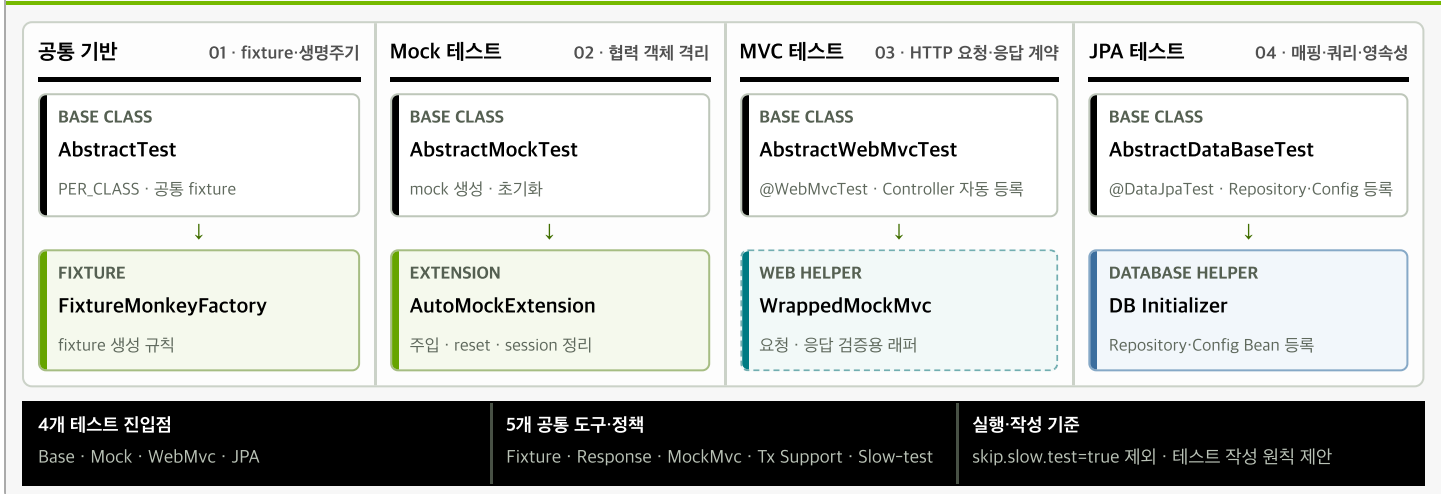


반복 테스트 준비를 test-support로 통합

테스트 진입점 4개와 공통 도구·정책 5개. fixture·mock 준비, 테스트 선택, CI 실행 기준 통합.

테스트 목적에 맞춰 공통 준비를 나눈 구조

진입점, 생명주기, 컨텍스트 범위, 실행 정책을 한 기준으로 묶었습니다.



코드와 사용 기준

테스트 진입점 · mock·컨텍스트 생명주기 · 별도 트랜잭션 · 느린 테스트 실행 기준 문서화

에이전트·스킬 패키징과 구조·행동 계약 검증

코드 탐색·설계 검토·테스트 절차를 에이전트 15개·스킬 32개로 분리. 구조 7개·행동 10개·설치 회귀 52개로 의존성과 계약 검증.

Agent가 역할을 정하고, skill이 절차를 제공하며, KB가 판단 근거를 보탭니다.

Agent Lab이 직접 작업을 실행하지는 않습니다. 자산과 의존성을 묶어 각 런타임에 설치합니다.

— 실행 순서 - - - 런타임 경계

ASSET REGISTRY · V0.1.0.0

manifest.txt

선택한 자산과 연결된 의존성을 한곳에서 찾습니다.

AGENTS

15

BUNDLES

14

SKILLS

32

설치된 자산을 런타임이 탐색·선택

AGENT + SKILL

01 · RUNTIME ROUTING

Agent 선택

작업 내용으로
고르거나 이름으로
호출

참조

02 · ROLE + KNOWLEDGE

정의와 지식 번들

.claude/agents
/
SOUL · principles ·
KB

호출

03 · PROCEDURE

연결된 skill 실행

.claude/skills
/
workflow · project
tools

workflows · project
tools

Claude Code

역할과 절차를 함께 설치

SKILL LANE

01 · DISCOVERY

Skill 탐색과 호출

/skills ·
\$skill-name

로동

02 · WORKFLOW

절차와 참고 지식

.agents/skills
/
SKILL.md · reference

실행

03 · PROJECT CONTEXT

현재 저장소에서 작업

file · shell · git · test

Codex

32/32 skill discovery 확인

SHARED EXECUTION

저장소 상태를 먼저 확인하고

코드·설정·도구를 기준으로 작업 범위를 정합니다.

OUTPUT CONTRACT

수정 근거와 검증 결과까지 남깁니다.

file:line · test result · risk · next action

01

자산 선택

agent · skill · all-*

02

연결 항목 확인

연결 skill을 재귀 탐색

03

런타임 경로에 설치

dry-run · apply · verify

04

회귀 검사

설치 52/52 · Codex 32/32

OpenCode

.opencode 경로 배치는 검사했습니다. agent 포맷 변환과 실제 선택·실행은 아직 검증하지 않았습니다.

현재 설치 범위

설치 범위 · Claude Code 에이전트·스킬, Codex 스킬 32개 · 메모리·자가 수정은 사람 승인 전제의 스키마·정책·평가 계약으로 제한

구현·판단의 공개 기록

기술 글 156편 · YouthCon 발표 · Spring OSS 개선 3건

YouthCon 2025 연사 '우리팀을 위한 Spring Test 만들기' 발표 · 테스트 코드 축소 기준과 팀 그라운드 룰 수립 과정	기술 글 156편 · 2026.07 기준 Java·JPA·테스트·SQL 튜닝 시행착오 기록 · 글또 10기 백엔드·인프라 빌리지 준비위원·연사 참여
Spring OSS · 개선 제안 3건 반영 Kafka · Boot · Data JPA · 불변 컬렉션 처리·하드코딩 제거·자원 정리 로직 개선 3건 반영	OpenSearch 한글 분석 플러그인 JavaCafe Elasticsearch의 한·영 자판 변환·초성·자모 검색을 OpenSearch 2.19·Kotlin으로 이식 · 예시 16개로 검증
화면에 없던 7만+ 건의 오류 S3 삭제·DB 잔존 데이터 6년간 7만+ 건 발견 · 정상 응답 뒤 원인을 좁힌 과정 공개	테스트로 찾은 약 200만 건 Mock 테스트가 놓친 JPA 변경 감지 오류를 실제 구현 테스트로 재현 · 보정 스크립트로 약 200만 건 수정